

1 Introduction to Database Systems

A **Database System** is an organized collection of data, typically stored and accessed electronically from a computer system, managed by a **Database Management System (DBMS)**. The DBMS provides an interface for users and applications to create, retrieve, update, and manage data efficiently, ensuring data integrity, security, and concurrency. Database systems are foundational to applications in business, science, finance, web services, and more.

1.1 Key Concepts

- **Data vs. Information:** Data is raw (e.g., numbers, strings); information is processed data (e.g., reports).
- **DBMS Role:** Manages data storage, querying, and administration (e.g., MySQL, PostgreSQL, Oracle).
- **Advantages:** Data consistency, scalability, backup/recovery, security, concurrent access.
- **Applications:** E-commerce (Amazon), banking, social media (X), scientific research, AI datasets.
- **As of 2025:** Trends include cloud-native databases (e.g., AWS Aurora), AI-driven query optimization, blockchain-based decentralized databases, and integration with quantum computing for specialized workloads.

1.2 History

- **1960s:** Hierarchical (IMS) and network models (CODASYL).
- **1970:** Edgar Codd's relational model; foundation of modern databases.
- **1980s–1990s:** SQL standardization, commercial DBMS (Oracle, DB2). Object-oriented databases emerge.
- **2000s:** NoSQL for scalability (MongoDB, Cassandra). Big data with Hadoop HDFS.
- **2010s:** Cloud databases (Google Cloud Spanner). NewSQL (CockroachDB) for scalability + ACID.
- **2020s:** AI integration for analytics, time-series databases (InfluxDB), and edge computing databases for IoT.

2 Database Models

The structure of data storage defines the database model.

2.1 Hierarchical Model

- **Structure:** Tree-like, parent-child relationships.
- **Pros:** Efficient for fixed hierarchies.
- **Cons:** Inflexible, no complex relationships.
- **Use:** Legacy systems (e.g., IBM IMS).

2.2 Network Model

- **Structure:** Graph-like, many-to-many relationships.
- **Pros:** More flexible than hierarchical.
- **Cons:** Complex navigation.
- **Use:** Early systems, now rare.

2.3 Relational Model

- **Structure:** Tables (relations) with rows (tuples) and columns (attributes). Based on Codd's principles.
- **Keys:**
 - **Primary Key:** Uniquely identifies rows.
 - **Foreign Key:** Links tables.
 - **Candidate/Alternate Keys:** Other unique attributes.
- **Pros:** Simple, flexible, SQL support, ACID compliance.
- **Cons:** Scalability for massive data.
- **Examples:** MySQL, PostgreSQL, Oracle.

2.4 Object-Oriented Model

- **Structure:** Data as objects (like OOP classes).
- **Pros:** Natural for applications with complex data (e.g., multimedia).
- **Cons:** Limited querying support.
- **Examples:** ObjectDB.

2.5 NoSQL Models

For unstructured/semi-structured data, scalability.

- **Key-Value:** Simple key-value pairs (e.g., Redis, DynamoDB).
- **Document:** JSON/BSON documents (e.g., MongoDB).
- **Column-Family:** Wide columns for analytics (e.g., Cassandra, HBase).
- **Graph:** Nodes/edges for relationships (e.g., Neo4j for social networks).
- **Pros:** Scales horizontally, flexible schemas.
- **Cons:** Weaker consistency (CAP theorem: pick Availability or Consistency in distributed systems).

2.6 NewSQL

- Combines relational ACID with NoSQL scalability (e.g., Google Spanner, CockroachDB).

2.7 Other Models

- **Time-Series:** For sequential data (e.g., InfluxDB for IoT).
- **Spatial:** For GIS (e.g., PostGIS).
- **Multimodel:** Supports multiple types (e.g., ArangoDB).

3 Relational Database Concepts

The relational model dominates due to its simplicity and robustness.

3.1 Schema and Instances

- **Schema:** Structure (tables, columns, constraints).
- **Instance:** Data at a moment (rows).

3.2 Integrity Constraints

- **Domain:** Valid values for columns (e.g., age ≥ 0).
- **Key:** Uniqueness of primary keys.
- **Referential:** Foreign keys match primary keys.
- **Entity:** No nulls in primary keys.

3.3 Normalization

Eliminates redundancy, ensures consistency.

- **1NF:** No repeating groups (atomic values).
- **2NF:** 1NF + non-key attributes depend on entire key.
- **3NF:** 2NF + no transitive dependencies.
- **BCNF:** Stronger 3NF; every determinant is a candidate key.
- **Higher NFs:** 4NF, 5NF for complex cases.
- **Denormalization:** Intentional redundancy for performance (e.g., data warehouses).

3.4 SQL (Structured Query Language)

- **DDL (Data Definition):** CREATE, ALTER, DROP (tables, schemas).
- **DML (Data Manipulation):** SELECT, INSERT, UPDATE, DELETE.
- **DCL (Data Control):** GRANT, REVOKE (permissions).
- **TCL (Transaction Control):** COMMIT, ROLLBACK.

Example (PostgreSQL):

```

1 CREATE TABLE Students (
2     id INT PRIMARY KEY,
3     name VARCHAR(50),
4     dept_id INT,
5     FOREIGN KEY (dept_id) REFERENCES Departments(id)
6 );
7 INSERT INTO Students VALUES (1, 'Alice', 101);
8 SELECT name FROM Students WHERE dept_id = 101;

```

4 Database Design

4.1 Phases

1. **Requirements Analysis:** Gather user needs (e.g., ER diagrams).
2. **Conceptual Design:** Model entities, relationships (ER/EER diagrams).
 - **Entities:** Objects (e.g., Student).
 - **Attributes:** Properties (e.g., name, id).
 - **Relationships:** Associations (e.g., Student TAKES Course).
 - **Cardinality:** 1:1, 1:N, N:M.
3. **Logical Design:** Map to relational schema (e.g., ER to tables).
4. **Physical Design:** Storage, indexes, partitioning.

4.2 ER Diagram Example

- Entity: Student(id, name), Course(id, title).
- Relationship: Enrolls(Student, Course, grade).
- Becomes tables: Students, Courses, Enrolls(student_id, course_id, grade).

5 Transaction Management

A **transaction** is a sequence of operations treated as a unit (e.g., bank transfer: debit A, credit B).

5.1 ACID Properties

- **Atomicity:** All or nothing.
- **Consistency:** Preserves database integrity.
- **Isolation:** Transactions don't interfere (serializability).
- **Durability:** Committed changes are permanent.

5.2 Concurrency Control

- **Problems:** Dirty read, non-repeatable read, phantom read.
- **Solutions:**
 - **Locks:** Shared (read), exclusive (write). 2PL (Two-Phase Locking).
 - **Timestamps:** Order transactions by time.
 - **MVCC (Multiversion Concurrency Control):** Snapshots (e.g., PostgreSQL).
- **Deadlocks:** Detect (wait-for graph) or prevent (timeout).

5.3 Recovery

- **Logging:** Undo/redo logs for crash recovery (e.g., ARIES algorithm).
- **Checkpoints:** Periodic snapshots to reduce recovery time.

6 Storage and Indexing

6.1 Storage

- **Physical:** Disk blocks, pages (typically 4KB).
- **Allocation:** Heap (unordered), sequential, clustered (by key).
- **File Structures:** B+ trees, hash tables for fast access.

6.2 Indexing

- **Primary Index:** On primary key (clustered).
- **Secondary Index:** On non-key attributes (unclustered).
- **B+ Tree:** Balanced tree, $O(\log n)$ search, ideal for ranges.
- **Hash Index:** $O(1)$ for exact matches, poor for ranges.
- **Bitmap Index:** For low-cardinality columns (e.g., gender).
- **Full-Text Index:** For search (e.g., Elasticsearch).

7 Query Processing and Optimization

7.1 Steps

1. **Parsing:** Validate SQL syntax.
2. **Optimization:** Choose best execution plan (cost-based).
3. **Execution:** Run plan (e.g., join algorithms).

7.2 Optimization Techniques

- **Query Rewriting:** Simplify expressions (e.g., push down selections).
- **Join Algorithms:** Nested loop, hash join, merge join.
- **Statistics:** Histograms, cardinality estimates for cost.
- **Indexes:** Use or skip based on cost.

Example: `SELECT * FROM Students WHERE age > 20` uses index on age if available.

8 Distributed Databases

For scalability across multiple nodes.

8.1 Concepts

- **Fragmentation:** Horizontal (by rows), vertical (by columns).
- **Replication:** Full/partial copies for availability.
- **CAP Theorem:** Consistency, Availability, Partition tolerance—pick two.
 - **CP:** Relational (e.g., Spanner).
 - **AP:** NoSQL (e.g., Cassandra).
- **Distributed Transactions:** Two-Phase Commit (2PC).

8.2 Challenges

- Latency, network partitions, consistency trade-offs.

9 NoSQL and Big Data

9.1 NoSQL Features

- **Schema-less:** Flexible for unstructured data.
- **Scalability:** Horizontal (sharding, replication).
- **BASE:** Basically Available, Soft state, Eventually consistent.

9.2 Big Data Systems

- **Hadoop:** HDFS + MapReduce for batch processing.
- **Spark:** In-memory, faster for iterative tasks.
- **Data Lakes:** Raw data storage (e.g., AWS S3 + Athena).

10 Advanced Topics

- **Cloud Databases:** Fully managed (AWS RDS, Azure Cosmos DB). Serverless (DynamoDB).
- **AI Integration:** Query optimization via ML (e.g., Oracle's Autonomous DB). AI analytics (e.g., vector databases for embeddings).
- **Blockchain Databases:** Immutable ledgers (e.g., BigchainDB).
- **Time-Series:** Optimized for IoT, finance (e.g., TimescaleDB).
- **Quantum Databases:** Emerging for quantum queries (research phase).
- **Privacy:** Homomorphic encryption, differential privacy.

11 Database Administration

- **Backup/Recovery:** Full, incremental, point-in-time recovery.
- **Performance Tuning:** Indexing, query optimization, caching.
- **Security:** Authentication (OAuth), encryption (AES), auditing.
- **Monitoring:** Tools like pgAdmin, Oracle Enterprise Manager.

12 Practical Implementation

Example (SQLite in Python):

```
1 import sqlite3
2 conn = sqlite3.connect('example.db')
3 cursor = conn.cursor()
4 cursor.execute('CREATE TABLE users (id INTEGER PRIMARY KEY, name TEXT)')
5 cursor.execute('INSERT INTO users (name) VALUES (?)', ('Alice',))
6 cursor.execute('SELECT * FROM users')
7 print(cursor.fetchall()) # [(1, 'Alice')]
8 conn.commit()
9 conn.close()
```

13 Best Practices

- **Design:** Normalize for OLTP, denormalize for OLAP.
- **Query:** Avoid SELECT *, use indexes wisely.
- **Scalability:** Partition large tables, use caching (Redis).
- **Security:** Sanitize inputs to prevent SQL injection.